

Cyber Solutions by Thales

Application Testing & Application Security

*Overview of the possibilities when
we combine both*



Dominique Righetto



Intrusion & Application Security team



Luxembourg Software Testing Event - Edition 2024

OPEN



THALES
Building a future we can all trust

Little reminder

| What is a security vulnerability?

- It's a bug having a Confidentiality and/or Integrity and/or Availability impact on the affected feature and its data.
- Like for development:
 - This type of bug can be detected via a test.
 - The later it is discovered, higher is the cost of its remediation in terms of time and/or money.

Little myth busting

■ “It requires dedicated tools”

- ✗ You can perform many types of security tests using same tools and technique daily used by a DevOps teams.

■ “It requires commercial tools”

- ✗ Free and Open-Source tools exist for a large type of security tests.
- 💡 Such tools can be used to “get started” into “application security testing” activities.

Little myth busting

■ “A security vulnerability is complex”

- ✗ Most critical security vulnerabilities are often caused by a simple lack of validation of an input received.

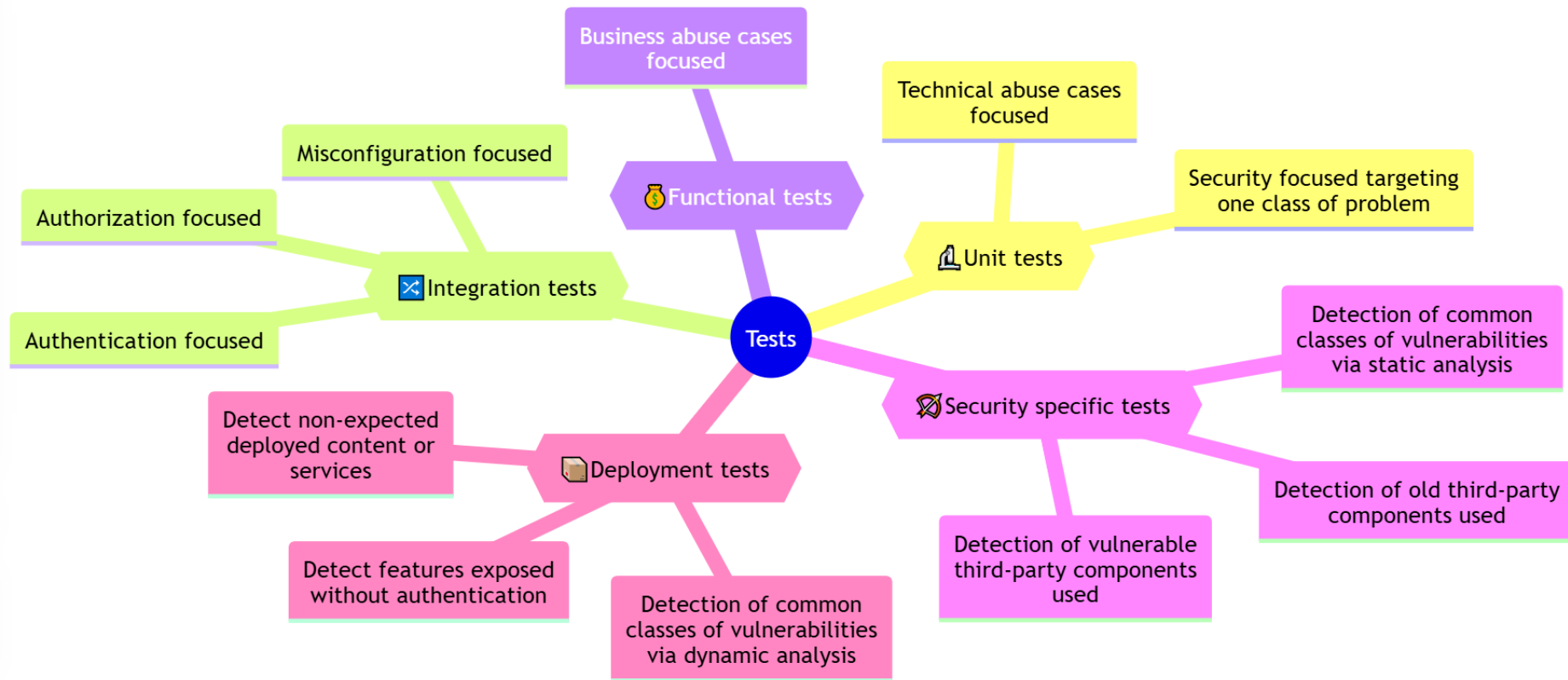
■ “Only a security consultant/[Insert LinkedIn title buzzword here] can find it”

- ✗ Once correctly explained, any developer can identify a security vulnerability.



"Security Testing" activities **require knowledge** and not tools:
Tools are only useful to scale the detection of security vulnerabilities!

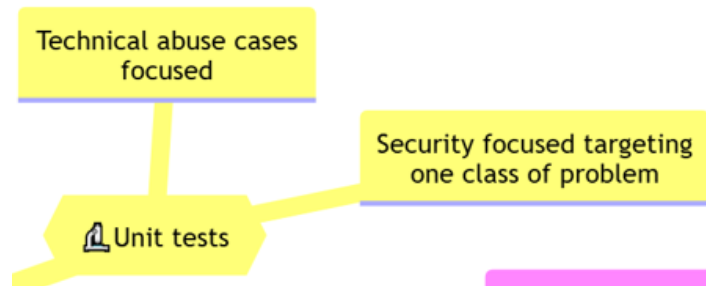
Overview of some possible types of security tests



Types of security tests

| Type

- **Unit** tests.



| Goal

- Validate that a feature is not prone to a **specific class** of security vulnerability.

| Examples of test cases

- Ensure that a feature, to parse an XML content, is not prone to XML external entity injection (XXE) or XML entity expansion (XEE) attacks.
- Ensure that a feature, to apply validation via regular expressions, is not prone to regular expression DOS attacks (called ReDOS).

Types of security tests

| Type

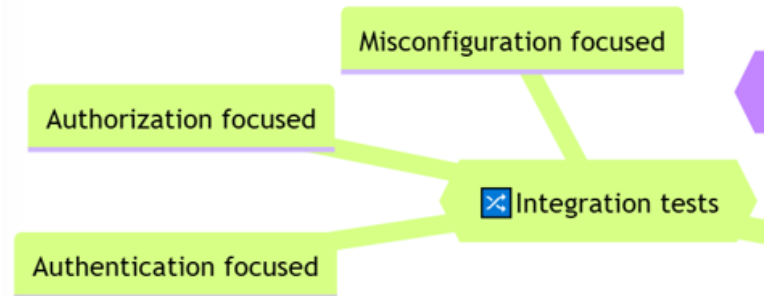
- **Integration** tests.

| Goal

- Validate that a feature behaves like expected, from a security perspective, at runtime once integrated into the whole systems.

| Example of test cases

- Ensure that the authorization rules are in sync according to the authorization matrix.
- Ensure that all the features require to be authenticated.



Types of security tests

| Type

- **Functional** tests.

| Goal

- Validate that a feature is not prone to abuse, when facing different kinds of malicious or unexpected input data, tailored for the business context.

| Example of test cases

- Ensure that a feature, determining the interest rate of a loan, is not prone to abuse, using the user-controlled information provided, as the attack vector.
- Ensure that a feature accepting images rejects any images containing hidden binary file.

Business abuse cases
focused



Functional tests

Types of security tests

| Type

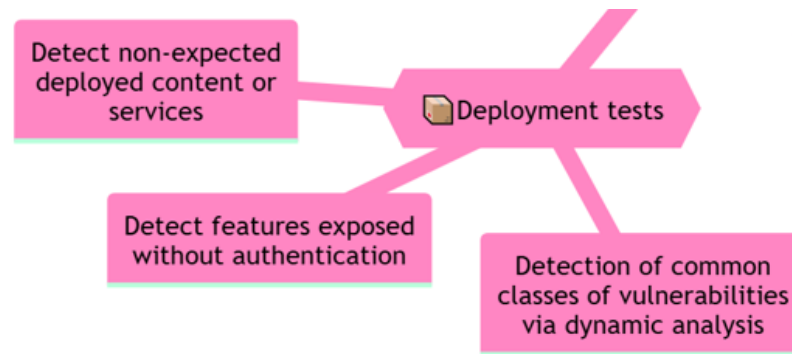
- **Deployment** tests.

| Goal

- Validate that a feature or a systems, once deployed, only expose the expected content or services.

| Example of test cases

- Detect unexpected deployed content, services, or configuration.
- Detect default configuration/setting/credentials in place.



Types of security tests

| Type

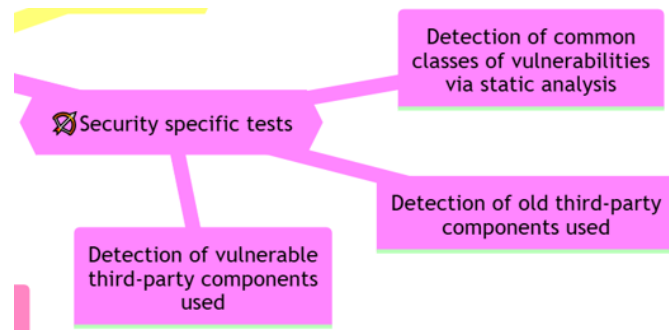
- **Security specific** tests.

| Goal

- Validate a particular aspect of a feature or a system from a security perspective.

| Example of test cases

- Identify common technical security vulnerabilities in the code base, during the implementation, via a tuned static code analyzer.
- Detect exposure to a particular vulnerability, like for example, Log4Shell.





Example of implementation n°1

Example of a unit test case for an XML parsing feature

➤ Sample business code

```
public static BankAccount loadBankAccountInformation(File xml) throws Exception {
    DocumentBuilderFactory dbfInstance = DocumentBuilderFactory.newInstance();
    DocumentBuilder builder = dbfInstance.newDocumentBuilder();
    Document doc = builder.parse(xml);
    Element root = doc.getDocumentElement();
    String name = root.getElementsByTagName("name").item(0).getTextContent();
    String iban = root.getElementsByTagName("iban").item(0).getTextContent();
    return new BankAccount(iban, name);
}
```

➤ Related unit test code

```
@Test
public void loadBankAccountInformation() throws Exception {
    String refIban = "DK9520000123456789";
    //Test evil case
    File evil = new File("data/xxe.xml");
    BankAccount account = AppCode.loadBankAccountInformation(evil);
    assertNotNull(account);
    assertEquals(refIban, account.getIban());
    //Test clean case
    //...
```



Example of implementation n°1

Example of a unit test case for an XML parsing feature

➤ Test input content

```
</> xxe.xml x
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!DOCTYPE data [ <!ENTITY xxe SYSTEM "evil.txt"> ]>
3  <account>
4    <name>Dominique Righetto</name>
5    <iban>&xxe;</iban>
6  </account>
```

```
evil.txt x
1  EVIL_CONTENT_LOADED
```

➤ Execution

```
✖ Tests failed: 1 of 1 test - 58 ms
C:\Workspace\Java\Jdk21\bin\java.exe

org.opentest4j.AssertionFailedError:
Expected :DK9520000123456789
Actual   :EVIL_CONTENT_LOADED
<Click to see difference>
```

OPEN



Example of implementation n°2

Example of a deployment test case to check for presence of map files

- To help with debugging, a source map files contain **essential information about how the compiled code maps** to the original code.
- Example of a JavaScript file and its associated map file content:

```
main.1553175d.js x
1  /*! For license information please see main.1553175d.js
2  (function(){var __webpack_modules__={3361:function(e,t,
function(){return ie}});var r=function(){function e(e){
function(e){var n;n=0===t.tags.length?t.insertionPoint?
t.prepend?t.container.firstChild:t.before:t.tags[t.tags
container.insertBefore(e,n),t.tags.push(e)},this.isSpee
speedy,this.tags=[],this.ctr=0,this.nonce=e.nonce,this.
```



```
main.1553175d.js.map x
1
2  "version": 3,
3  "file": "static/js/main.1553175d.js",
4  "mappings": ";wGAQdA,IAAIA,EAAOB,WAE5B,SAASA,EAAWC,GAC1B,IAAIC,EAAQC,KAEZA,KAAKC,WAAa,SAAUc",
5  "sources": [
6    "../node_modules/@emotion/sheet/dist/emotion-sheet.browser.esm.js",
7    "../node_modules/stylis/src/Utility.js",
8    "../node_modules/stylis/src/Tokenizer.js",
9    "../node_modules/stylis/src/Enum.js",
10   "App.tsx",
11   "index.tsx"
12 ],
13 "sourcesContent": [
14   "/*\n\nBased off glamor's StyleSheet, thanks Sunil ❤️\n\nhigh performance StyleSheet fo\n15   /**\n * @param {number} n * @return {number}\n */\nexport var abs = Math.abs\n16   \"import {from, trim, charat, strlen, substr, append, assign} from './Utility.js'\n17   \"import {IMPORT, LAYER, COMMENT, RULESET, DECLARATION, KEYFRAMES} from './Enum.js'\n18   \"import {COMMENT, RULESET, DECLARATION} from './Enum.js'\n19   \"import {abs, charat, trim, fr\n20 ],\n21 \"names\": [\n22   \"StyleSheet\",\n23   \"options\",\n24   \"this\",\n25   \"AppLayout\",\n26   \"App\"\n27 ],\n28 \"sourceRoot\": \"\"
```

OPEN



Example of implementation n°2

Example of a deployment test case to check for presence of map files

```
main.1553175d.js.map X
1
2   "version": 3,
3   "file": "static/js/main.1553175d.js",
4   "mappings": ";wGAqDA,IAAIA,EAA0B,WAE5B,SAASA,EAAWC,GAC1B,IAAIC,EAAQC,KAEZA,KAAKC,WAAa,SAAU",
5   "sources": [
6     "../../node_modules/@emotion/sheet/dist/emotion-sheet.browser.esm.js",
7     "../../node_modules/stylis/src/Utility.js",
8     "../../node_modules/stylis/src/Tokenizer.js",
9     "../../node_modules/stylis/src/Enum.js",
10    "App.tsx",
11    "index.tsx"
12  ],
13  "sourcesContent": [
14    "/*\n\nBased off glamor's StyleSheet, thanks Sunil ♡\n\nhigh performance StyleSheet for",
15    "/*\n * @param {number}\n * @return {number}\n */\nexport var abs = Math.abs\n\n**\n",
16    "import {from, trim, charat, strlen, substr, append, assign} from './Utility.js'\n\nexpe",
17    "import {IMPORT, LAYER, COMMENT, RULESET, DECLARATION, KEYFRAMES} from './Enum.js'\n\nimpe",
18    "import {COMMENT, RULESET, DECLARATION} from './Enum.js'\n\nimport {abs, charat, trim, fro",
19  ],
20  "names": [
21    "StyleSheet",
22    "options",
23    "_this",
24    "AppLayout",
25    "App"
26  ],
27  "sourceRoot": ""
28
```



Example of implementation n°2

Example of a deployment test case to check for presence of map files

➤ Script created

```
#!/bin/bash
TARGET="https://[redacted]"
WORK_FILE="/tmp/urls.txt"
USER_AGENT="Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.0.0 Safari/537.36"
# Extract all JS bundles composing the Single Page Application
# See https://github.com/projectdiscovery/katana
katana -silent -duc -em js -u $TARGET > $WORK_FILE
# Check if a map file is referenced in every JS bundle
rc=0
for url in $(cat $WORK_FILE)
do
    # Extract the name of the JS bundle
    js_bundle=$(echo $url | awk -F '/' '{print $NF}')
    # Check if the keyword is present
    map_file_referenced=$(curl -A "$USER_AGENT" -sk $url | grep -Fc "sourceMappingURL=")
    if [ $map_file_referenced -ne 0 ]
    then
        # Try a call to the map file to see if a valid map file is returned
        curl --user-agent "$USER_AGENT" --silent --insecure "$url.map" | jq -r ".sourcesContent | length" 2>/dev/null 1>&2
        if [ $? -eq 0 ]
        then
            echo "[!] Map file found for JS bundle '$js_bundle' => $url.map"
            rc=1
        fi
    fi
done
exit $rc
```



Example of implementation n°2

Example of a deployment test case to check for presence of map files

➤ Script created

```
# Extract all JS bundles composing the Single Page Application  
# See https://github.com/projectdiscovery/katana  
katana -silent -duc -em js -u $TARGET > $WORK_FILE
```




Example of implementation n°2

Example of a deployment test case to check for presence of map files

➤ Script created

```
# Check if a map file is referenced in every JS bundle
rc=0
for url in $(cat $WORK_FILE)
do
    # Extract the name of the JS bundle
    js_bundle=$(echo $url | awk -F '/' '{print $NF}')
    # Check if the keyword is present
    map_file_referenced=$(curl -A "$USER_AGENT" -sk $url | grep -Fc "sourceMappingURL=")
    if [ $map_file_referenced -ne 0 ]
    then
        # Try a call to the map file to see if a valid map file is returned
        curl --user-agent "$USER_AGENT" --silent --insecure "$url.map" | jq -r ".sourcesContent | length" 2>/dev/null 1>&2
        if [ $? -eq 0 ]
        then
            echo "[!] Map file found for JS bundle '$js_bundle' => $url.map"
            rc=1
        fi
    fi
done
```



Example of implementation n°2

Example of a deployment test case to check for presence of map files

➤ Execution

```
[Temp] $ bash search_for_map_files.sh
[!] Map file found for JS bundle 'main.1553175d.js' ⇒ https://[redacted]/static/js/main.1553175d.js.map
[Temp] $ echo $?
1
[Temp] $
```

My personal golden rules

Be and stay pragmatic:

- Refactor tests to ensure to always bring added value to a DevOps team.
- Ask to the DevOps team if the material provided is relevant and helpful for them.
- Adopt the same approach/mindset than a DevOps team to implements security tests.

Knowledge is the key, not the tools:

- Any tool used must allow a DevOps team/me to learn about the security vulnerability addressed.
- If a vulnerability is commonly found, then present it to the team in a fun and kind way.

Security must be the most transparent possible to a DevOps team:

- The DevOps role already includes a huge number of tasks and required knowledge. So, every time it is possible, do not bother the team with security aspects: **Made it automatic and invisible → Concept of “secure by default”.**

Key points to take away

Start small regarding the security tests:

- Leverage past security audits to build your custom security tests.
- Start with a vulnerability commonly found into your apps.
- Grow based on your failures: It is allowed and very informative to fail 😊
- Require any external security auditor to provide effective/actionable technical material for any vulnerability mentioned in their audit report.

Use free and open-source tools prior to buy commercial ones:

- Start by identify your needs and objectives in term of results!
- Free and open-source tools are used as a foundation.
- This groundwork will be useful to define a test plan for the evaluation of commercial tools if needed later.

Thanks for your time and attention

■  Any questions?

■  Several resources, on the presented topic, are provided in next slides.

■  **Excellium Services S.A.**

- >  contact@excellium-services.lu
- >  <https://excellium-services.com>
- >  <https://excellium-services.com/blog>

Resources

Unit tests

- <https://github.com/righettod/code-snippets-security-utils>

Integration tests

- https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Testing_Automation_Cheat_Sheet.html
- <https://github.com/righettod/voxxeddays-lux-2018>

Functional tests

- https://cheatsheetseries.owasp.org/cheatsheets/Abuse_Case_Cheat_Sheet.html

Resources

Deployment tests

- <https://github.com/righettod/toolbox-pentest-web/blob/master/scripts/identify-hidden-content-webapi.sh>
- <https://github.com/righettod/toolbox-pentest-web/blob/master/scripts/identify-hidden-content-website.sh>
- <https://github.com/righettod/toolbox-pentest-web/blob/master/scripts/identify-hidden-jslibs.sh>
- <https://github.com/righettod/toolbox-pentest-web/blob/master/scripts/identify-js-packages-from-map-files.py>

Resources

| Security specific tests

- > <https://github.com/righettod/toolbox-codescan>
- > <https://github.com/righettod/toolbox-jwt>

| Training

- > <https://pentesterlab.com/>
- > <https://portswigger.net/web-security>
- > <https://www.appsecengineer.com/>

Resources

Tools

- <https://github.com/danielmiessler/SecLists>
- <https://github.com/ffuf/ffuf>
- <https://github.com/projectdiscovery/nuclei>
- <https://github.com/ovh/venom>
- <https://semgrep.dev/explore>